TP1 — Déploiement Proxmox avancé : templates, clones, snapshots et restauration

Contexte

Vous êtes l'équipe virtualisation d'une PME.

L'entreprise souhaite déployer rapidement plusieurs serveurs Linux pour ses services internes :

- un serveur web ;
- un serveur de base de données ;
- une machine d'administration ;
- une machine de test permettant de valider les snapshots et les restaurations.

Votre objectif est de préparer une base Proxmox propre, réutilisable et industrialisable.

Aucun rendu écrit n'est demandé.
Vous devez simplement **me montrer votre travail.**

Définitions à connaître avant de commencer

Proxmox VE

Proxmox VE est une plateforme de virtualisation open source qui permet d'administrer des machines virtuelles, des conteneurs, du stockage, du réseau, des sauvegardes et des clusters depuis une interface web.

Dans les TP précédents sur **KVM/QEMU/libvirt**, vous avez travaillé sur les briques de base de la virtualisation Linux.
Dans ce TP, vous allez retrouver ces mêmes briques, mais intégrées dans une plateforme d'exploitation plus complète.

Proxmox repose principalement sur :

- **KVM** pour l'accélération matérielle des machines virtuelles ;
- **QEMU** pour créer et exécuter le matériel virtuel ;

- **LXC** pour les conteneurs Linux ;
- **Linux Bridge / VLAN** pour le réseau virtuel ;
- **LVM-thin, ZFS, Directory, NFS, iSCSI ou Ceph** pour le stockage ;
- **Corosync** pour le cluster ;
- **vzdump** pour les sauvegardes.

À retenir :

> Dans les TP KVM, vous manipuliez les composants séparément.
> Dans Proxmox, vous utilisez une plateforme qui orchestre ces composants.

Hyperviseur

Un **hyperviseur** est une couche logicielle qui permet d'exécuter plusieurs systèmes d'exploitation sur une même machine physique.

Dans les TP précédents, vous avez vu que **KVM transforme Linux en hyperviseur** grâce aux extensions matérielles du processeur :

- Intel VT-x ;
- AMD-V.

Dans Proxmox, l'hyperviseur utilisé pour les VMs est toujours **KVM**.

La différence est que Proxmox ajoute au-dessus :

- une interface web ;
- une gestion centralisée des VMs ;
- une gestion du stockage ;
- une gestion réseau ;
- des snapshots ;
- des sauvegardes ;
- des fonctions de cluster et de haute disponibilité.

KVM

KVM, pour *Kernel-based Virtual Machine*, est le module de virtualisation intégré au noyau Linux.

Il permet à Linux d'exécuter des machines virtuelles avec une performance proche du matériel physique.

Dans vos TP KVM précédents, vous avez vu que KVM :

- utilise les extensions CPU Intel VT-x ou AMD-V ;
- fournit l'accélération matérielle ;
- permet d'exécuter des systèmes invités Linux ou Windows ;
- travaille avec QEMU pour créer une VM complète.

Dans Proxmox, KVM reste la base technique des machines virtuelles.

À retenir :

> KVM ne fournit pas seul toute la VM.
> Il fournit surtout l'accélération matérielle.
> QEMU construit et pilote la machine virtuelle.

QEMU

QEMU est le composant qui crée et exécute la machine virtuelle.

Il gère le matériel virtuel :

- CPU virtuel ;
- mémoire ;
- disque ;
- carte réseau ;
- contrôleur disque ;
- BIOS ou UEFI ;
- périphériques virtuels.

Dans vos TP précédents, la formule utilisée était :

> **QEMU crée et pilote la VM ; KVM la rend rapide.**

Dans Proxmox, lorsque vous créez une VM depuis l'interface web, Proxmox génère en réalité une configuration QEMU/KVM.

Vous ne lancez pas QEMU directement à la main : Proxmox le fait pour vous.

---

libvirt

libvirt est une couche d'administration qui permet de gérer des machines virtuelles à travers une API et des outils.

Dans vos TP KVM, libvirt servait à administrer les VMs avec :

- `virt-manager` ;

- `virsh` ;
- des connexions comme `qemu:///system`.

Dans Proxmox, la logique est différente.
Proxmox n'utilise pas virt-manager comme interface principale. Il fournit :

- sa propre interface web ;
- sa propre API ;
- ses propres commandes de gestion ;
- ses propres fichiers de configuration.

Correspondance générale :

Dans les TP KVM	Dans Proxmox
virt-manager	Interface web Proxmox
virsh	commandes Proxmox comme `qm`
libvirt	couche d'administration Proxmox
fichiers XML libvirt	fichiers de configuration Proxmox
gestion manuelle	orchestration centralisée

À retenir :

> Proxmox joue le rôle de console d'administration complète pour KVM/QEMU, avec plus de fonctions intégrées que virt-manager.

Machine virtuelle

Une **machine virtuelle** est une machine complète simulée par l'hyperviseur.

Elle possède :

- son propre système d'exploitation ;
- son disque virtuel ;
- sa mémoire ;
- ses interfaces réseau ;
- son BIOS ou UEFI ;
- ses périphériques virtuels.

Dans les TP KVM, vous avez créé ou analysé des VMs en comprenant les composants internes : disque, réseau, CPU, mémoire, cycle de vie, stockage et snapshots.

Dans ce TP Proxmox, vous allez refaire la même logique, mais via une plateforme plus proche d'un environnement de production.

Une VM Proxmox est donc toujours une VM QEMU/KVM, mais administrée par Proxmox.

Conteneur LXC

Un **conteneur LXC** est une forme de virtualisation légère.

Contrairement à une VM, un conteneur ne possède pas son propre noyau.
Il partage le noyau Linux de l'hôte Proxmox.

Différence essentielle :

VM KVM/QEMU	Conteneur LXC
Noyau indépendant	Noyau partagé avec l'hôte
Isolation plus forte	Isolation plus légère
Plus lourd	Plus rapide et plus léger
Peut exécuter Linux, Windows, BSD	Principalement Linux
Simule une machine complète	Isole des processus

Dans ce TP, vous travaillez surtout sur les VMs, car l'objectif est de faire le lien avec les TP KVM.

QEMU Guest Agent

Le **QEMU Guest Agent** est un agent installé à l'intérieur de la VM.

Il permet à Proxmox de mieux communiquer avec le système invité.

Il sert notamment à :

- afficher l'adresse IP de la VM dans Proxmox ;
- arrêter proprement la VM ;
- récupérer certaines informations système ;
- améliorer la cohérence des snapshots et des sauvegardes ;
- faciliter l'administration depuis l'hyperviseur.

Lien avec les TP KVM :

Dans un TP KVM classique, vous pouvez démarrer une VM sans agent invité, mais l'hôte voit moins précisément ce qui se passe à l'intérieur.
Dans Proxmox, cet agent devient important car la plateforme doit administrer proprement les VMs depuis l'interface web.

À retenir :

> Le QEMU Guest Agent est le pont de communication entre Proxmox et l'intérieur de la VM.

VirtIO

VirtIO est un ensemble de pilotes paravirtualisés utilisés avec KVM/QEMU.

Il permet à la VM d'utiliser des périphériques optimisés au lieu de périphériques entièrement émulés.

Exemples :

- carte réseau VirtIO ;
- disque VirtIO ;
- contrôleur SCSI VirtIO ;
- ballooning mémoire.

Dans vos TP KVM, VirtIO permettait déjà d'améliorer les performances des VMs.
Dans Proxmox, il faut aussi le privilégier pour les VMs de production.

Comparaison :

Type de périphérique	Performance	Usage
IDE	Faible	Compatibilité ancienne
E1000	Moyenne	Compatibilité réseau
VirtIO	Élevée	Production KVM/Proxmox
SCSI VirtIO	Élevée	Disques VM modernes

À retenir :

> En environnement KVM/Proxmox, VirtIO est généralement le choix recommandé pour les performances.

Template

Un **template** est une VM modèle.

Elle ne sert pas à être utilisée directement.
Elle sert à créer rapidement d'autres VMs à partir d'une base propre.

Dans les TP KVM, vous avez probablement créé des VMs une par une.
Dans Proxmox, on industrialise cette logique avec des templates.

Un bon template doit être :

- minimal ;
- propre ;
- à jour ;
- équipé de QEMU Guest Agent ;
- prêt à être cloné ;
- sans configuration trop spécifique.

Exemple :

tmpl-debian-12-base

À retenir :

> Le template est une VM de référence.
> Il sert à standardiser les déploiements.

Clone complet

Un **clone complet** est une copie indépendante d'une VM ou d'un template.

Il possède son propre disque complet.

Avantages :

- indépendant du template ;
- plus robuste ;
- peut être déplacé plus facilement ;
- moins risqué pour une VM importante.

Inconvénients :

- consomme plus d'espace ;
- plus long à créer.

Lien avec KVM :

Dans un environnement KVM classique, cela revient à créer une copie complète du disque d'une VM, par exemple un disque QCOW2 ou RAW indépendant.

Dans Proxmox, cette opération est simplifiée par l'interface web.

Clone lié

Un **clone lié** est une VM qui dépend du disque parent du template.

Il ne stocke que les différences par rapport au disque de base.

Avantages :

- très rapide à créer ;
- économise beaucoup d'espace ;
- pratique pour des labs, des tests ou des salles de TP.

Inconvénients :

- dépend du template parent ;
- moins autonome ;
- risque si le disque parent est supprimé ou corrompu ;
- moins adapté à certaines VMs critiques.

Lien avec KVM :

Dans KVM/QEMU, cette logique ressemble aux images différentielles QCOW2 avec backing file.

À retenir :

> Un clone lié est pratique pour les TP et les déploiements rapides, mais il introduit une dépendance technique.

Snapshot

Un **snapshot** est un point de restauration d'une VM à un instant donné.

Il permet de revenir en arrière après :

- une mauvaise configuration ;
- une mise à jour ratée ;
- une panne volontaire ;
- une erreur d'administration.

Dans vos TP KVM, vous avez déjà rencontré la logique de snapshot au niveau du disque ou de la VM.
Dans Proxmox, cette fonction est intégrée à l'interface et dépend du type de stockage utilisé.

Attention :

> Un snapshot n'est pas une sauvegarde.

Un snapshot dépend généralement du stockage d'origine.
Si le stockage est perdu, le snapshot est perdu aussi.

Rollback

Un **rollback** est l'action de revenir à un snapshot précédent.

Exemple :

1. Vous créez un snapshot avant une modification.
2. Vous cassez volontairement la configuration réseau.
3. La VM ne répond plus.
4. Vous effectuez un rollback.
5. La VM revient à l'état précédent.

Lien avec les TP KVM :

Dans KVM, vous avez pu voir le cycle de vie d'une VM : création, démarrage, arrêt, modification, snapshot, restauration.
Dans Proxmox, le rollback rend ce cycle plus simple à démontrer.

Sauvegarde

Une **sauvegarde** est une copie exportée d'une VM ou d'un conteneur vers un stockage de backup.

Elle est différente d'un snapshot.

Snapshot	Sauvegarde
Point de restauration local	Copie exportée
Rapide	Plus long

Dépend souvent du stockage source	Peut être restaurée ailleurs
Utile avant modification	Utile pour PRA/PCA
Ne remplace pas un backup	Protection réelle contre perte VM

Lien avec les TP KVM :

Dans un environnement KVM manuel, il faut souvent gérer soi-même l'export ou la copie des disques.
Dans Proxmox, la sauvegarde est intégrée avec `vzdump` et l'interface web.

Stockage `local`

Le stockage **local** est généralement un stockage de type directory.

Il sert souvent à stocker :

- les ISO ;
- les templates de conteneurs ;
- les fichiers de sauvegarde ;
- parfois des disques VM selon la configuration.

Lien avec KVM :

Dans un TP KVM classique, cela correspond à un répertoire local contenant des fichiers comme :

- images ISO ;
- disques QCOW2 ;
- fichiers RAW ;
- fichiers de configuration.

Dans Proxmox, ce stockage est déclaré et géré depuis l'interface.

Stockage `local-lvm`

Le stockage **local-lvm** est souvent un stockage de type LVM-thin.

Il sert principalement à stocker les disques des VMs et des conteneurs.

Avantages :

- gestion efficace de l’espace ;
- création rapide de volumes ;
- snapshots possibles selon configuration ;
- adapté aux disques VM.

Lien avec KVM :

Dans les TP KVM, les disques étaient souvent vus comme des fichiers.
Avec LVM-thin, les disques des VMs sont plutôt gérés comme des volumes logiques.

À retenir :

> `local` sert souvent aux fichiers comme les ISO.
> `local-lvm` sert souvent aux disques des VMs.

Bridge réseau

Un **bridge réseau** est un commutateur virtuel.

Il permet de connecter :

- l’hôte Proxmox ;
- les VMs ;
- les conteneurs ;
- éventuellement le réseau physique.

Dans vos TP KVM, vous avez vu que le réseau virtuel peut être en NAT, bridge ou réseau isolé.
Dans Proxmox, le bridge Linux est la base du réseau VM.

Exemple :

vmbr0

Un bridge Proxmox peut servir à :

- connecter les VMs au LAN ;
- créer un réseau interne ;
- transporter des VLAN ;
- séparer management, serveurs, DMZ et stockage.

À retenir :

> Le bridge Proxmox joue le rôle de switch virtuel pour les VMs.

Cycle de vie d’une VM

Le **cycle de vie d’une VM** correspond aux différentes étapes de son existence :

- création ;
- installation ;
- démarrage ;
- arrêt ;
- modification ;
- snapshot ;
- clonage ;
- sauvegarde ;
- restauration ;
- suppression.

Dans les TP KVM, vous avez étudié ce cycle avec les outils KVM/QEMU/libvirt.
Dans ce TP Proxmox, vous devez montrer que vous maîtrisez le même cycle, mais dans une plateforme centralisée.

Synthèse

Notion vue en TP KVM	Équivalent ou prolongement dans Proxmox
KVM	Hyperviseur utilisé par Proxmox
QEMU	Moteur d’exécution des VMs Proxmox
libvirt / virt-manager	Remplacés par l’interface web, l’API et les outils Proxmox
disque QCOW2 / RAW	Disque VM sur local, local-lvm, ZFS, NFS, Ceph
réseau NAT / bridge	vmbr, VLAN, firewall Proxmox
snapshot manuel	snapshot intégré Proxmox
copie de disque	clone complet
backing file QCOW2	clone lié
sauvegarde manuelle	vzdump / backup Proxmox
administration locale	gestion centralisée Proxmox

Objectif du TP

Ce TP fait le lien entre les séances précédentes sur KVM/QEMU/libvirt et l'exploitation d'une plateforme de virtualisation complète.

Proxmox ne remplace pas KVM : il l'utilise.

L'objectif est donc de comprendre comment les concepts déjà vus — VM, disque virtuel, bridge, snapshot, clone et cycle de vie — sont industrialisés dans une interface d'administration centralisée.

À la fin du TP, vous devez être capables de :

- préparer un hôte Proxmox ;
- créer une VM Linux propre ;
- installer et activer QEMU Guest Agent ;
- transformer une VM en template ;
- créer des clones complets et liés ;
- personnaliser plusieurs VMs ;
- créer et restaurer un snapshot ;
- provoquer une panne puis restaurer l'état fonctionnel.

Contraintes

Vous travaillez en binôme.

Aucun rendu écrit n'est demandé.

Vous devrez uniquement me **montrer votre travail** à la fin du TP.

Toutes les machines doivent être fonctionnelles et testables.

Partie 1 — Préparation de l'hôte Proxmox

Travail demandé

Vous devez vérifier et configurer les éléments suivants :

- le nom d'hôte Proxmox ;
- l'adresse IP de management ;
- la passerelle ;

- le serveur DNS ;
- le stockage disponible ;
- l'accès à l'interface web ;
- les dépôts Proxmox ;
- la présence d'au moins une ISO Linux.

Éléments à vérifier

Dans l'interface Proxmox, vous devez identifier :

- le nœud Proxmox ;
- le stockage `local` ;
- le stockage `local-lvm` ou équivalent ;
- l'espace disque disponible ;
- les ISO disponibles ;
- les informations réseau de l'hôte ;
- l'état général du nœud.

Résultat attendu

Votre hôte Proxmox doit être opérationnel et prêt à héberger des VMs.

Vous devez être capables de montrer :

- l'interface web Proxmox accessible ;
- au moins une ISO Linux disponible ;
- le stockage destiné aux ISO ;
- le stockage destiné aux disques VM ;
- la configuration réseau de management ;
- l'état général du nœud sans erreur critique.

Partie 2 — Création d'une VM Linux propre

Travail demandé

Vous devez créer une VM Linux qui servira de base au futur template.

Configuration minimale attendue :

Élément	Valeur attendue

CPU	2 vCPU
RAM	2 Go
Disque	20 Go
Carte réseau	VirtIO
Contrôleur disque	SCSI VirtIO
OS	Debian ou Ubuntu Server
SSH	Activé
Utilisateur admin	Créé

Nom attendu

Vous pouvez utiliser un nom clair comme :

debian-template

ou :

ubuntu-template

Configuration attendue

La VM doit utiliser :

- un disque sur le stockage prévu pour les VMs ;
- une carte réseau en VirtIO ;
- un contrôleur disque SCSI VirtIO ;
- une adresse IP fonctionnelle ;
- un accès SSH fonctionnel ;
- un utilisateur administrateur.

Résultat attendu

Votre VM de base doit démarrer correctement.

Vous devez pouvoir montrer :

- la configuration matérielle de la VM ;
- le système Linux installé ;
- l’adresse IP de la VM ;
- la connexion SSH fonctionnelle ;

- l’accès administrateur ;
- le bon fonctionnement réseau.

Partie 3 — Installation de QEMU Guest Agent

Travail demandé

Dans la VM Linux, vous devez installer et activer **QEMU Guest Agent**.

Vous devez ensuite activer l’option correspondante dans la configuration Proxmox de la VM.

Résultat attendu

Dans l’interface Proxmox, la VM doit afficher correctement :

- son adresse IP ;
- son état ;
- les informations remontées par l’agent invité.

Éléments à montrer

Vous devez montrer :

- que QEMU Guest Agent est installé dans la VM ;
- que le service est actif ;
- que l’option QEMU Guest Agent est activée côté Proxmox ;
- que Proxmox récupère correctement l’adresse IP de la VM.

Partie 4 — Nettoyage et préparation du template

Travail demandé

Avant de convertir la VM en template, vous devez la nettoyer et la préparer.

Vous devez vérifier :

- que le système est à jour ;
- que le service SSH fonctionne ;
- que QEMU Guest Agent fonctionne ;
- que le hostname est générique ;
- que la VM ne contient pas de configuration trop spécifique ;
- que la VM peut servir de base à plusieurs clones.

Objectif

Le template doit être suffisamment propre pour permettre la création de plusieurs VMs différentes sans conflit.

Vous devez éviter de laisser :

- un nom de machine trop spécifique ;
- une configuration réseau incohérente ;
- des services inutiles ;
- des fichiers temporaires inutiles ;
- une configuration qui empêcherait le clonage propre.

Résultat attendu

Votre VM doit être prête à devenir un modèle réutilisable.

Partie 5 — Conversion en template

Travail demandé

Vous devez convertir la VM préparée en **template Proxmox**.

Le template doit être clairement nommé.

Exemple :

tmpl-debian-12-base

ou :

tmpl-ubuntu-server-base

Résultat attendu

Le template doit apparaître dans l’interface Proxmox.

Vous devez pouvoir montrer :

- la VM convertie en template ;
- le nom du template ;
- les options de clonage disponibles ;
- le fait que le template sert de base aux futures VMs.

Partie 6 — Création de clones

Travail demandé

À partir du template, vous devez créer plusieurs VMs.

Vous devez créer :

- 1 clone complet ;
- 2 clones liés ;
- 1 VM supplémentaire destinée aux tests de snapshot et rollback.

Configuration attendue :

VM	Type de clone	Rôle
`web01`	Clone complet	Serveur web
`db01`	Clone lié	Serveur base de données
`admin01`	Clone lié	Machine d'administration
`test-rollback01`	Au choix	VM de test snapshot

Résultat attendu

Les quatre VMs doivent exister dans Proxmox.

Vous devez pouvoir montrer :

- les VMs créées ;
- le type de clone utilisé ;
- les VMs démarrées ;
- l'espace disque consommé ;

- la différence entre le clone complet et le clone lié ;
- la dépendance éventuelle des clones liés au template.

Partie 7 — Personnalisation des VMs

Travail demandé

Chaque VM doit être personnalisée.

Chaque machine doit avoir :

- un hostname différent ;
- une adresse IP différente ;
- un accès SSH fonctionnel ;
- un rôle identifiable.

Configuration attendue :

VM	Rôle	Résultat attendu
`web01`	Serveur web	Page web accessible
`db01`	Base de données simulée	Service actif ou port ouvert
`admin01`	Administration	SSH vers les autres VMs
`test-rollback01`	Test de panne	Snapshot puis restauration

Travail spécifique sur `web01`

Sur `web01`, vous devez installer un serveur web simple.

Le service web doit être accessible depuis une autre machine.

Test attendu :

curl http://IP_DE_WEB01

Le serveur doit répondre avec une page ou un message identifiable.

Exemple de contenu attendu :

Serveur web01 opérationnel

Travail spécifique sur `db01`

Sur `db01`, vous devez simuler un serveur de base de données.

Vous pouvez utiliser :

- MariaDB ;
- PostgreSQL ;
- un service TCP simple ;
- ou un service applicatif léger.

Le service doit être actif et identifiable.

Travail spécifique sur `admin01`

Depuis `admin01`, vous devez pouvoir joindre les autres VMs.

Vous devez vérifier :

- l'accès réseau vers `web01` ;
- l'accès réseau vers `db01` ;
- l'accès SSH si autorisé ;
- le test HTTP vers `web01`.

Résultat attendu

Vous devez montrer :

- les hostnames différents ;
- les adresses IP différentes ;
- les accès SSH ;
- le service web fonctionnel ;
- le service de `db01` actif ;
- la communication entre les VMs.

Partie 8 — Snapshot, panne volontaire et rollback

Travail demandé

Vous devez créer un snapshot de la VM `test-rollback01`.

Ensuite, vous devez provoquer volontairement une panne.

Exemples de panne possibles :

- suppression d'un fichier de configuration ;
- désactivation du SSH ;
- arrêt d'un service important ;
- modification réseau incorrecte ;
- suppression d'une page web ;
- mauvaise configuration du serveur web.

La panne doit être réelle et visible.

Étapes attendues

Vous devez suivre cette logique :

1. La VM fonctionne.
2. Vous créez un snapshot.
3. Vous provoquez une panne.
4. Vous montrez que la panne existe.
5. Vous effectuez un rollback.
6. Vous montrez que la VM fonctionne à nouveau.

Résultat attendu avant rollback

Avant le rollback :

- la panne doit être visible ;
- le service ou la configuration doit être cassé ;
- vous devez pouvoir constater le dysfonctionnement.

Résultat attendu après rollback

Après le rollback :

- la VM doit revenir à l'état fonctionnel ;
- le service doit répondre ;
- la configuration doit être restaurée ;
- le retour arrière doit être visible dans Proxmox.

Validation finale

À la fin du TP, vous devez me montrer:

- l'hôte Proxmox fonctionnel ;
- le stockage utilisé ;
- l'ISO Linux ;
- la VM de base ;
- QEMU Guest Agent actif ;
- le template ;
- les clones complets et liés ;
- les VMs personnalisées ;
- le service web fonctionnel ;
- la VM `db01` avec un service actif ;
- la VM `admin01` utilisée pour les tests ;
- un snapshot ;
- une panne volontaire ;
- un rollback réussi ;
- le retour à un état fonctionnel.

Résultat final attendu

Votre environnement final doit contenir au minimum :

Élément	État attendu
Hôte Proxmox	Fonctionnel
ISO Linux	Disponible
VM de base	Installée et propre
QEMU Guest Agent	Installé et actif
Template	Créé
`web01`	Fonctionnelle avec serveur web
`db01`	Fonctionnelle avec service actif
`admin01`	Fonctionnelle pour les tests
`test-rollback01`	Snapshot + panne + rollback

Clone complet	Présent et identifié
Clones liés	Présents et identifiés